How the seven FoC services talk over Pub/Sub, and how each is containerized

A four-page teaching guide to the CS3219 P32 repository, 19 October 2026.[1]

1 Two ways to talk: phone calls and mailing lists

Phone calls (HTTP). Everything a user does starts as a synchronous call: the browser calls the Nuxt server, which forwards to *one* backend with the user's JWT and waits for the answer. Backends never call each other over HTTP. Each one checks the JWT itself with the shared `JWT_SECRET`.

Mailing lists (Pub/Sub). Everything that other services need to learn happens asynchronously. A service *publishes* a message to a **topic**, which is like a mailing list. Each interested reader owns a **subscription**, which is like a personal mailbox on that list. Pub/Sub keeps a copy of every message in every mailbox until the reader **acknowledges** it ("got it"). There are two ways to read a mailbox:

- *Push:* Pub/Sub POSTs each message to the reader's HTTP endpoint. A 2xx answer is the acknowledgement; any other status, or a timeout, means "try again later". Every consumer here uses push, except notification-service.
- *Pull:* the reader fetches messages and calls `ack()` or `nack()`. Notification-service pulls, as a long-running process that also holds browser WebSockets.

Two rules shape the rest of the design. First, Pub/Sub puts a message only into subscriptions that *already exist* when it is published. Second, delivery is *at least once*: a message can arrive twice or late, so every handler checks whether it has already applied a message (it is *idempotent*). Messages on a subscription with ordering enabled arrive in publish order per **ordering key**, which here is the user, supplier or order ID.

Fanout versus competing consumers. Several subscriptions on one topic each get every message: that is *independent fanout*, and it is how one sign-up reaches four services. Several processes reading the *same* subscription share its messages, so each message goes to only one of them: that is *competing consumers*. Notification-service uses both on `order-events`. Every running instance creates its own `notification-<uuid>` subscription, so all instances see every event and can update the browsers connected to them. All instances also share `notification-store`, so each notification is stored once. A subscription can also carry a **filter** on message attributes, so unwanted messages never reach it.

There are six topics. Four carry *events*, which report facts that already happened: `user-events`, `supplier-events`, `order-events`, and `credit-events` for credit results. The other two carry *commands*, which ask one service to do something (`credit-commands`, `order-commands`). In the normal workflow only the saga orchestrator sends commands. Figure 1 shows every topic and every subscription.

2 The seven services and what each sends and receives

frontend-service (Nuxt). The only thing the browser loads. Its server routes keep the JWT in an httpOnly cookie and call the backends; it owns no database and uses no Pub/Sub.

user-service (API + `user-event-worker`). Handles sign-up, login and profiles, and signs the JWTs. Unusually, the API does not write users to its database: it publishes `user.created`, `.updated` or `.deleted`, carrying a full user snapshot and a global `version`, and the worker writes the `users` table from those events.

supplier-service (API + `supplier-event-worker` + ticker). Admin edits to suppliers are published to `supplier-events` through an outbox. The worker keeps a local copy of users (for role checks) from `user-events` and offers a "publish leftover outbox rows" route that the ticker calls every minute.

order-service (API + ticker). Owns orders and their state machine. It keeps local copies of users and of every supplier version, so it can check an order without calling anyone. Every state change is a row in `order_events`, which is both the order history and its outbox, and is published to `order-events`. It obeys saga commands from `order-commands`.

[1] Grounded in the code at `ea16eec` (`main`); the service code is unchanged at `7e8c31b`, which adds only the event bus tap and these docs. File and line evidence for every claim is in `CLAIM_MAP.md`; the full 33-page version, with payload examples and Cloud Run details, is `guide-detailed.pdf` in the same folder. Nothing was run to write this guide; flows described as working come from code reading and earlier logged runs on the local emulator.

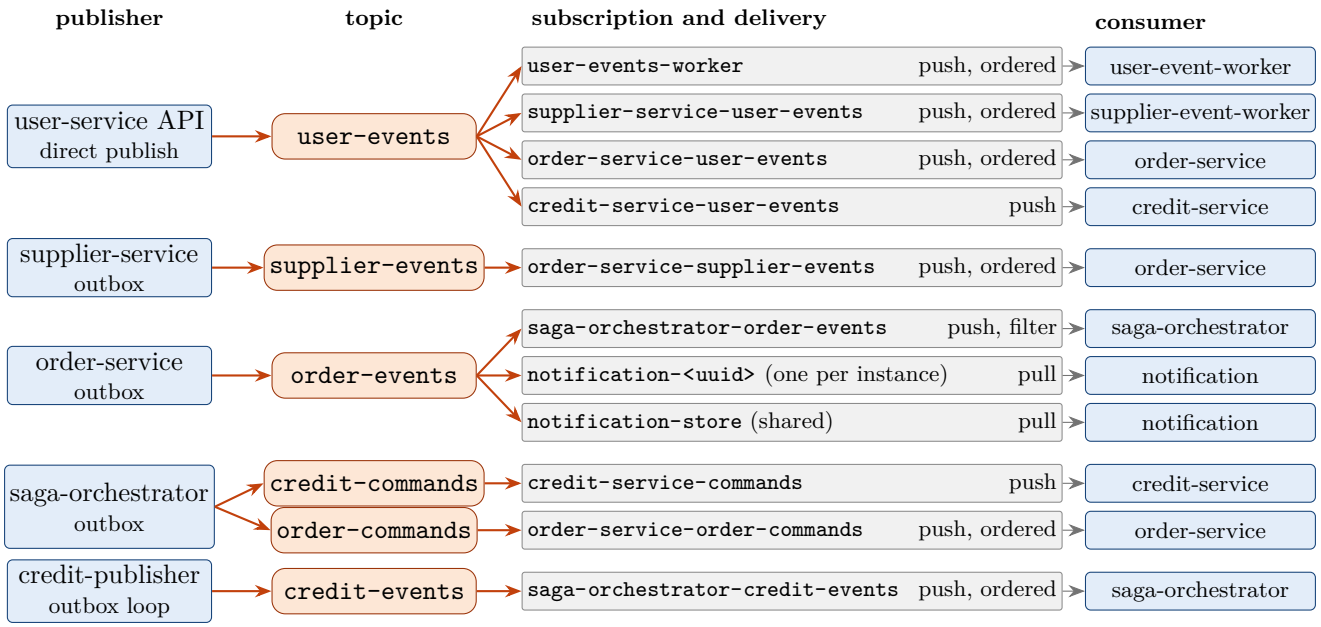


Figure 1: Every topic and subscription. "Ordered" means message ordering is enabled. The saga's filter passes only `order.created`, `.cancelled`, `.expired` and `.completed`; both notification subscriptions accept every `order.*` type.

saga-orchestrator (+ ticker). No user API. It reacts to four order event types and to credit results, keeps one saga row per order, and sends commands through its own outbox. Its ticker aborts reservations that credit has not answered within 60 seconds.

credit-service (API + `credit-publisher` + one-shot provisioner). Holds balances. It receives `user-events` (to open an account with 1000 credits) and `credit-commands` (reserve, release, transfer), and writes each result to `credit_outbox`. A separate publisher loop sends those rows to `credit-events` about once a second.

notification-service. Pulls `order-events`, pushes live updates to browsers over a WebSocket (`/ws`), and stores per-user notifications. Nobody is notified about their own action.

Each service owns one PostgreSQL database, and no service reads another's. That is why services learn about each other only through events, and why events carry whole snapshots rather than IDs.

3 Publishing reliably and acknowledging safely

The outbox. A service cannot commit a database change and publish to Pub/Sub as one atomic step. If it tried to do both, a crash in between would lose the message or announce a change that never happened. So supplier, order, saga and credit write the message as an *outbox row in the same database transaction* as the business change. A relay then publishes unpublished rows in order and marks them `published_at`. It runs right after the commit, at startup and on an every-minute sweep (credit's relay is its own process). An advisory lock allows one relay per database, and a key that fails to publish holds back its later rows, so per-key order holds. A crash after publishing but before marking the row publishes the same row again with the same ID. So the guarantee is "every committed change is announced *at least* once", never exactly once.

The exception: user-service. It publishes *instead of* writing its database. If the publish fails, the request fails and nothing changes. If it succeeds, the user row appears only after `user-event-worker` applies the event, so a login can briefly lag.

What counts as an acknowledgement. Order-service, the saga and the supplier worker answer **204** on success and also for a malformed message, which can never succeed and would otherwise block its ordering key. They answer **500** when the database fails, so Pub/Sub retries later. `user-event-worker` and credit-service answer **400** to malformed input. Pub/Sub retries that too, because no dead-letter topic is configured. Credit answers **503** on purpose while waiting for a prerequisite, for example an order's reservation before its user's account exists. Pub/Sub's redelivery then acts as the wait loop. Notification-service always acks its live subscription, but acks the store one only after saving.

How duplicates become harmless. User copies apply an event only if its `version` is newer than the stored one. Supplier versions and notifications insert with "on conflict do nothing" on a natural key. The saga records each `eventId` in `processed_messages`. Order-service refuses a second command with the same `commandId`. Credit stores every command under its `requestId` and replays the stored result.

4 End to end: sign-up, supplier edits and the order saga

Sign-up replicates the user to four places. `POST /register` validates the input and reserves a user ID and event version. It publishes `user.created` (key = user ID) and returns a JWT. Fanout then delivers the event independently, in any order, to four subscriptions. `user-event-worker` writes the login row. The supplier and order workers each upsert a small copy (id, display name, role) if the version is newer. Credit opens an account with 1000 credits. Until its copy exists, order-service answers *503 "your account is still being set up"*, and credit shows its balance as being set up. Deletion soft-deletes the copies, so old orders can still refer to the user.

A supplier edit reaches order-service. An admin's `PUT /suppliers/:id` updates the row and inserts a `supplier.updated` outbox row with the full snapshot in one transaction. The relay publishes it (key = supplier ID). Order-service *appends* it to `supplier_versions` and never edits old rows. New orders use the latest version, and existing orders keep the version they were placed against.

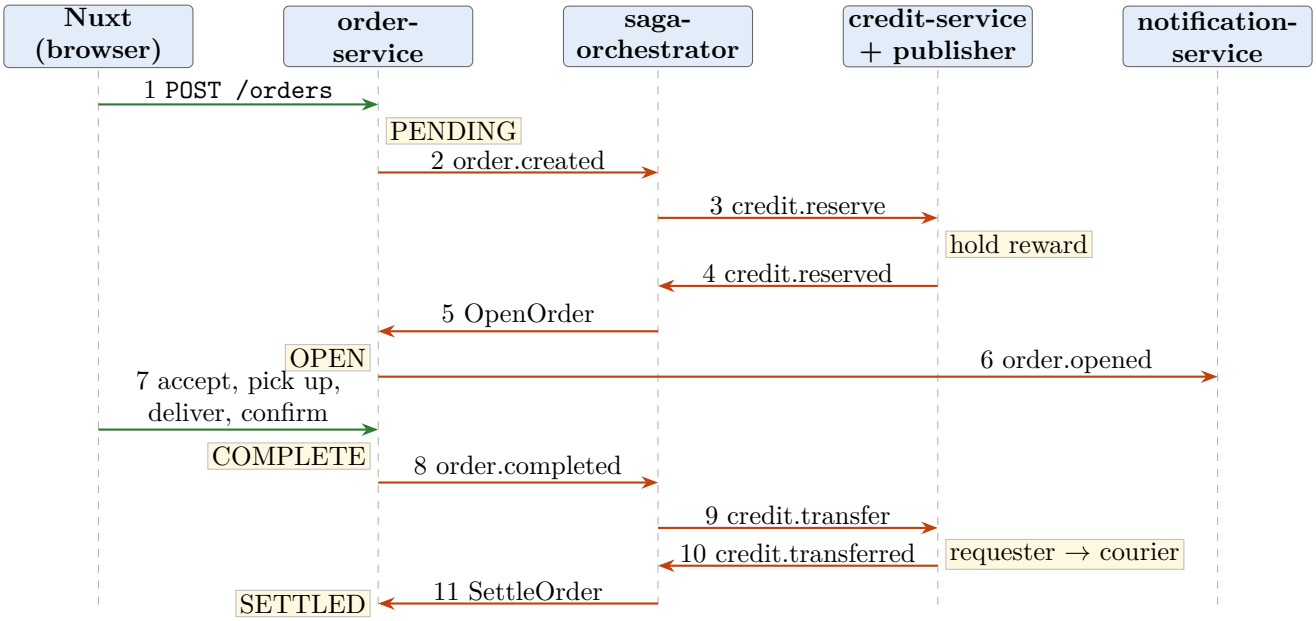


Figure 2: The order saga. Green arrows are HTTP calls; orange arrows are Pub/Sub messages through the topics in Figure 1. Notification-service sees every `order.*` event, not only the one drawn.

The order saga (Figure 2). A *saga* is a business process spread over several services. Each step is a local transaction, and a later failure is undone by a *compensating* step rather than a global rollback. (1) The requester's order is stored as `PENDING`, and `order.created` goes into the outbox. (2) The saga creates its row in `RESERVING`, with a 60-second deadline, and (3) sends `credit.reserve`. Credit checks that the available credits cover the reward and holds them. (4) Its publisher emits `credit.reserved`. The saga matches the `requestId`, moves to `OPEN` and (5) sends `OpenOrder`. Order-service opens the order for two hours, and (6) `order.opened` reaches notification-service, which notifies the requester. (7) A courier accepts, picks up and delivers, and the requester confirms, all over HTTP. The saga's filter skips the intermediate events. (8) On `order.completed` the saga (9) sends `credit.transfer`. Credit moves the held credits to the courier, (10) reports success, and (11) `SettleOrder` marks the order `SETTLED`, which notifies the courier.

When it goes wrong. If credits are short, credit answers `credit.reservation_rejected`, the saga sends `RejectOrder`, and nothing needs undoing. If credit is down, the command waits in its subscription. After 60 seconds the saga ticker compensates: it sends `credit.release` *and* `AbortOrder`. When credit returns, either queued command may run first, because that subscription is unordered. Either way the credits end up released, thanks to credit's per-order lock and reservation state. A cancelled or expired

order makes the saga send `credit.release`. If credit refuses a release or a transfer, the saga stops in `FAILED` with only a log line; nothing retries it.

5 How each service is containerized

The common pattern. Every service folder is its own Docker build context, so no image can contain another service's code. Supplier, order, saga and notification share a three-stage Dockerfile on `node:22-bookworm-slim`. *tooling* installs all dependencies and holds the TypeScript sources and migrations. *build* compiles to `dist/` and drops dev dependencies. *runtime* copies only `dist/` and production modules, runs as the unprivileged `node` user, and starts `node dist/index.js`. `package*.json` is copied before the sources, so the slow `npm ci` layer is cached when only code changes. Compose reuses images in two ways: `target: tooling` builds the one-shot `*-db-migrate` containers, and `command:` replaces the image's default command to start a worker or publisher from the same image. Table 1 lists what differs per service.

Service	Image	Port → host	Extra containers from the same image
frontend	build → runtime; ships only Nuxt's `.output`; user `node`; backend URLs read at start from `NUXT_*` env	3000 → 3001	none
user	one stage; all deps; runs TypeScript with `tsx`; root	3333 → 3333	worker (`npm run worker`, 3334, unpublished); migrate (`drizzle-kit push`)
supplier	common three-stage	3000 → 3000	worker (`node dist/worker.js`, 3004, unpublished); migrate; `tools`-profile seeder
order	common three-stage	3000 → 3002	migrate
saga	common three-stage	3000, none	migrate
credit	two stages on `node:22-alpine`; reinstalls prod deps; root	3000 → 3004	publisher; one-shot `pubsub:provision`; migrate (from runtime image)
notification	common three-stage	3000 → 3003	migrate

Table 1: Per-service differences. Supplier, order and saga each also get a tiny `curlimages/curl` ticker: a shell loop of `sleep 60; curl -X POST` to an internal sweep route.

Networking. `compose.yaml` declares no networks, so every container joins one default network where each *service name* is a DNS name. Containers therefore reach each other at addresses like `http://order-service:3000`, `order-db:5432` and `pubsub-emulator:8085`, always on the *container* port. Publishing (`"3002:3000"`) is a separate door from the host, and `EXPOSE` only documents a port and opens nothing. For example, the supplier worker listens on 3004 although its image says `EXPOSE 3000`, and *host* port 3004 is credit-service. The browser uses only host ports 3001 (frontend) and 3003 (the notification WebSocket). The emulator and the saga are not published at all. Locally the emulator cannot sign push requests, so push URLs carry a local-only shared secret instead of Google's OIDC token.

Startup order encodes the subscription rule. The emulator starts empty and keeps everything in memory. When `PUBSUB_EMULATOR_HOST` is set, each consumer creates its own topics and subscriptions at startup, and only then starts listening. The provisioner does this for credit. So a passing `/health` check also means "my subscriptions exist". Compose uses `depends_on` with `service_healthy`, which waits for that check, and with `service_completed_successfully`, which waits for a migration or provisioning job to exit cleanly. As a result, user-service waits for its worker and for the supplier and order consumers, supplier-service waits for order-service, and the saga waits for order-service and the credit provisioner. In each case the readers exist before the first message is published. One gap: user-service does not wait for the credit provisioner. Restarting only the emulator loses every subscription until the consumers restart.

Persistence. Six `postgres:17` containers, one per service, keep `/var/lib/postgresql/data` in named volumes (`order-db-data`, ...). They survive `docker compose down`, are deleted by `down -v`, and are published only on `127.0.0.1` (5433–5439). The emulator has no volume, so undelivered messages die with it. `compose.dev.yaml` adds hot reloading by syncing `src/` into `tooling`-stage containers running `tsx watch`. The opt-in `compose.tap.yaml` adds the event bus tap on the same network, where it can reach the unpublished emulator. In Google Cloud the same images run on Cloud Run with real Pub/Sub, set up by `deploy.sh`; that transport is configured but has no recorded run.